

Supplier invoice exceptions

Checking purchase-order evidence before the next handoff

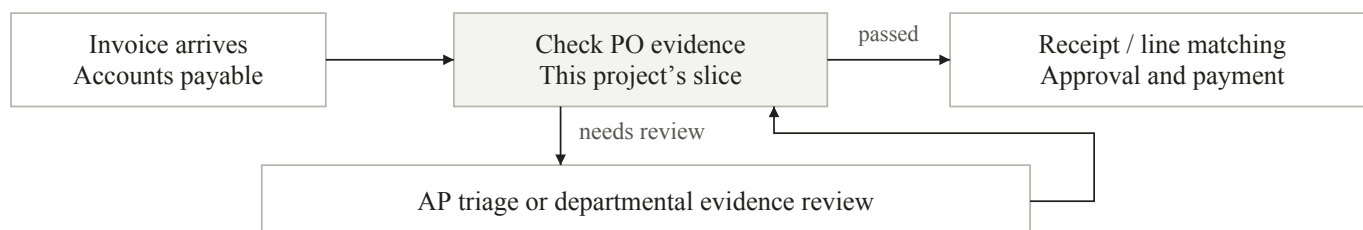
Mohit Kumar · 24bcs10222

Where invoices get stuck

Buckinghamshire Healthcare NHS Trust (BHT) reports **275 purchase-order (PO) lines raised after invoice date** in June 2026, versus 214 in May. It describes accounts payable (AP) staff following up with departments to arrange PO creation, and delays identifying the responsible department. [1] These are line counts, not an invoice exception rate. I examine **which PO evidence needs investigation and who should review it**.

Reference workflow

Requisition → authorised PO → supply / receipt



Simplified from public descriptions [1, 3]. The Trust's exact systems and handoffs are unverified.

Who would use this

Assumed roles and goals. No stakeholder interviews have taken place.

Role	Goal and constraint
AP manager · proposed owner	Choose which PO problems at arrival to investigate, within staffing limits.
AP processor	Find evidence for review and the responsible role. Cannot authorise a purchase.
Department buyer / budget holder	Confirm the reference or approval evidence within delegated authority.
Supplier · affected party	Predictable payment. Cannot repair the buyer's internal records.

Why start here

This handoff is documented and small enough to test end to end. Reference and approval evidence are inputs to fuller invoice matching. But an additional queue may add nothing to the existing procurement system. Earlier PO setup or better receipt capture could matter more. April's report attributes an increase in registered invoices mainly to delayed annual PO setup. [2] **I would change course if the existing report already provides equivalent evidence, or another blocker consumes more review time.**

Scope and success

The prototype checks PO references, supplier and request IDs, currency and approval evidence for routine GBP invoices with positive amounts. It identifies cases for human review. Capital, emergency, PO-exempt and credit documents are excluded. Order capacity, line matching, receipts, disputes, tax and payment decisions are outside scope. **Header checks alone do not authorise payment.**

KPI = confirmed PO-header exceptions at invoice receipt ÷ assessable eligible arrivals (passed + exceptions). UNKNOWN means insufficient or ambiguous evidence and is excluded from this denominator. Five supporting measures track open cohort exceptions by reason, gross value, median invoice age, routing-role coverage and assessment coverage. Success means correct, traceable decisions and repeatable output. A lower synthetic rate is not evidence of improvement.

Unknowns. Current tooling, actual workload, ownership authority and additional benefit remain unverified. Operational data and policies are synthetic. BHT is not a client.

[1] BHT, June 2026 finance report, pp. 15-16. [2] BHT, April 2026 finance report, p. 15. [3] HMRC, *Procure to pay*, purchase-order, receipt and invoice controls. Sources linked. Research checked 18 September 2026.

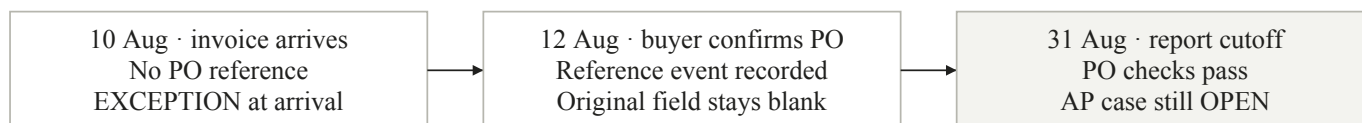
The data workflow

I generated synthetic SQL and CSV exports (seed 20260918): 60 ordinary examples and 40 named edge cases. Public reports lack linked invoice histories. This mix tests rules, not Trust performance.

Source of truth in the simulation	Record structure and key fields
Procurement · SQLite via SQL	PO → request → department; supplier_id, currency, created_at. Status events carry effective_at (event time) and recorded_at (entry time).
AP · invoice CSV	One invoice record: invoice_id, supplier_id, supplier invoice number, original PO/request; issue and arrival time; GBP pence.
AP · case-event CSV	One event: event_id, invoice_id, actor and two timestamps; follow-up, reference confirmation, closure or reopening.

SQL + CSV → preserve raw files → validate → model → metrics and review

Case INV-CASE-REFERENCE-REPAIR · synthetic



Several invoices can share a PO. Reducing histories to one invoice assessment before SQL joins prevents double-counting. Goods-receipt, allocation and payment events are absent.

Handling incomplete records

The run preserves raw bytes and stops on invalid hashes, schemas, procurement master records or incomplete declared histories. Whitespace, code case and timezones are normalised and logged. Extra copies of identical invoice and event rows are removed. Conflicting or malformed rows are quarantined. Evidence gaps that prevent a header assessment remain UNKNOWN. Approvals recorded after invoice receipt do not prove readiness when the invoice arrived. Future events cannot change the cutoff assessment.

Invoice rows: 102 raw = 97 retained + 1 duplicate removed + 4 quarantined.

After 7 exclusions, 90 remain: 42 passed, 37 exceptions and 11 UNKNOWN at invoice receipt.

Observed output from the synthetic run

Arrival cohort: 1 August 2026 00:00 to 31 August 2026 18:00 UTC, inclusive. Current review uses the same cutoff and cohort, not the whole AP backlog.

Measure	Result and interpretation
KPI · arrival exception rate	37 / 79 = 46.84% . Assessable excludes 11 UNKNOWN records.
Current open exceptions by reason	28 : missing reference 9; unapproved 7; cancelled 7; five other reasons 1 each.
Their gross invoice value	£42,454.05 . This is not unpaid value or loss.
Their median invoice age	21.33 days since invoice receipt, not time spent in exception.
Their routing-role coverage	23 / 28 = 82.14% . Five need AP to identify an owner first.
Arrival assessment coverage	79 / 90 = 87.78% . UNKNOWN stays in this denominator.

The queue has 37 items: 28 confirmed exceptions + 9 UNKNOWN. AP can use it to choose the next check. The arrival KPI helps the AP manager investigate upstream failures. Neither result measures a benefit.

Reliability and reproduction

51 tests pass, including checks against 40 separately specified scenarios. Reruns reproduce identical model, metric and review files. If an input hash check fails, the run records FAILED, keeps rejected raw data and retains the last successful output. Restoring the input reproduces the same run.

Run `python3 run.py run`, then `python3 run.py verify`.

Working repository and saved evidence: github.com/mohit-1710/fde-invoice-evidence

Python 3.9+, offline. README links raw snapshots, logs, tests, results and the walkthrough.